

Tarea 7. Integración MAUI, LLM, IOT

Programación de Inteligencia Artificial
CE en Inteligencia Artificial y Big Data

Antonio Ferrer Martínez

Abril 2026

Índice de contenidos

1. Introducción.....	4
2. Objetivos	4
3. Arquitectura general.....	5
4. Modelo LLM utilizado y configuración	6
4.1 LM Studio.....	6
4.2 Elección del modelo	7
4.3 System prompt	8
5. Configuración de la herramienta (tool).....	9
6. Aplicación .NET MAUI.....	14
6.1 Por qué MVVM y por qué CommunityToolkit	14
6.2 Estructura del proyecto	14
6.3 LmStudioService: cómo habla la app con el modelo.....	16
6.4 MainViewModel: lo que pasa al darle a Enviar	18
6.5 Comandos por voz.....	20
6.6 Interfaz de usuario.....	20
7. Node-RED	22
7.1 Por qué Node-RED.....	22
7.2 El flujo.....	23
7.3 Broker MQTT.....	23
8. ESP32 y conexionado	24
8.1 Hardware y por qué PlatformIO.....	24
8.2 Conexionado.....	25
8.3 Setup: WiFi, MQTT y OLED.....	27
8.4 Reconexión MQTT	28

8.5 Callback MQTT: cuando llega un comando	29
8.6 PWM, parpadeo sin delay, preset semáforo y OLED	31
9. Servidor MCP	33
10. Instalación y puesta en marcha	35
10.1 Requisitos previos	35
10.2 Instalar y arrancar LM Studio	36
10.3 Instalar y arrancar Node-RED	37
10.4 Cargar firmware en la ESP32	38
10.5 Compilar y ejecutar la app MAUI	39
10.6 Orden de arranque	39
11. Conclusiones	40
12. Bibliografía y referencias	41
12.1 Modelo LLM y function calling	41
12.2 .NET MAUI y MVVM	42
12.3 Node-RED y MQTT	42
12.4 ESP32 y hardware	42
12.5 MCP (Model Context Protocol)	43
12.6 Accesibilidad y formato del documento	43

1. Introducción

Esta tarea plantea construir una aplicación en la que el usuario interactúa con un modelo de lenguaje (LLM) ejecutado en local, y ese modelo, a través del mecanismo de function calling, termina controlando un dispositivo físico. En mi caso el dispositivo es una placa ESP32 con tres LEDs y una pantalla OLED, y el modelo puede pedirle que encienda, apague, regule o haga secuencias con esas luces.

El flujo es en cadena: la app MAUI envía el mensaje del usuario al LLM; si la respuesta es una llamada a herramienta, el resultado se manda por HTTP a un flujo Node-RED; Node-RED publica el comando por MQTT al ESP32; y el ESP32 aplica los estados a los LEDs y devuelve una confirmación por otro topic MQTT. Todo el procesamiento de lenguaje natural corre en local, sin servicios externos.

Además de la app, el mismo control se expone a través de un servidor MCP (Model Context Protocol) en un proyecto .NET aparte, de forma que clientes como Claude Desktop pueden invocar la misma herramienta sin pasar por la interfaz gráfica.

2. Objetivos

Los objetivos que me he marcado en el proyecto son:

- Ejecutar un modelo de lenguaje en local sin depender de servicios en la nube.
- Aprender a usar function calling con modelos pequeños (menos de 1B de parámetros) y ver sus limitaciones.
- Construir una aplicación .NET MAUI multiplataforma con patrón MVVM.

- Integrar una pasarela Node-RED y el protocolo MQTT como capa entre la app y el hardware.
- Diseñar el firmware de la ESP32 para que acepte combinaciones de estados complejas (brillo, parpadeos a distintas velocidades, secuencias y código morse) sin bloquear el bucle principal.
- Exponer el mismo control mediante un servidor MCP compatible con clientes externos.
- Dejar todo documentado y reproducible por otra persona con las instrucciones de este documento.

3. Arquitectura general

El sistema tiene cinco piezas que se comunican en cadena: la aplicación MAUI (cliente), LM Studio (servidor local del modelo), Node-RED (pasarela HTTP a MQTT), un broker MQTT público y la ESP32 (dispositivo).

La app es el único punto de contacto con el usuario. Cuando el usuario escribe o dicta algo, la app lo manda a LM Studio a través de su API compatible con OpenAI, junto con la definición de la herramienta controlar_luces. Si el modelo responde con una llamada a esa herramienta, la app interpreta los parámetros y envía por POST un JSON a Node-RED. Node-RED empaqueta ese JSON y lo publica en un topic MQTT al que la ESP32 está suscrita. El firmware parsea el JSON y aplica los estados a los LEDs mediante PWM, y opcionalmente dispara secuencias automáticas (semáforo, ola, morse). Por último, la ESP32 publica en un segundo topic (ACK) un resumen de lo que ha aplicado.

En paralelo, el proyecto LucesIoT.MCP expone la misma herramienta a través del protocolo MCP sobre STDIO, reutilizando el mismo flujo Node-RED → MQTT → ESP32. Así el ESP32 recibe órdenes indistintamente desde la app o desde cualquier cliente MCP.

Arquitectura del Sistema

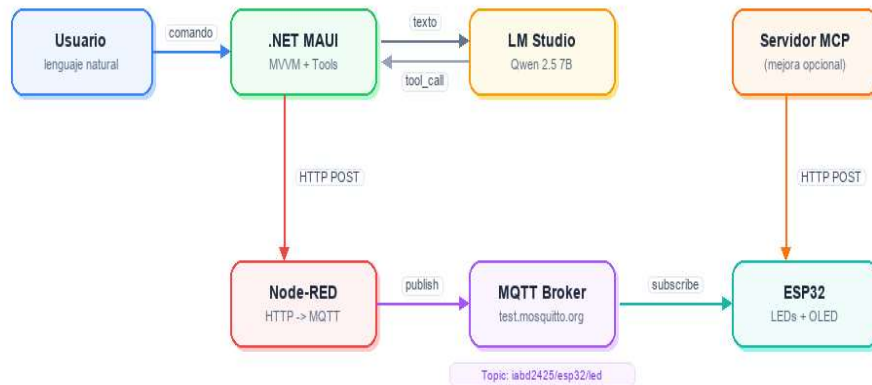


Figura: arquitectura general del sistema (cliente, LLM, pasarela, dispositivo)

4. Modelo LLM utilizado y configuración

4.1 LM Studio

Para correr el LLM en local he usado LM Studio (<https://lmstudio.ai>). La gracia es que levanta una API compatible con la de OpenAI en localhost:1234, así que desde la app hago peticiones HTTP normales como si fuera OpenAI de verdad, pero todo se queda en mi ordenador sin mandar datos a ningún servidor externo.

Miré también Ollama (<https://ollama.com>), que hace lo mismo y tiene buena documentación, pero LM Studio tiene interfaz gráfica para gestionar modelos, cambiar parámetros y ver logs en tiempo real. Eso para ir probando va mucho mejor. Además lo había usado en tareas anteriores y ya me movía bien por él.

4.2 Elección del modelo

En esta parte me tiré un buen rato investigando. El ejemplo del enunciado usaba un modelo genérico con 2 herramientas (on/off) pero no decía cuál. Yo empecé con Qwen 2.5 7B Instruct, que es bastante popular para llamadas a funciones. Funcionaba, pero en mi equipo iba lentísimo. Varios segundos para cada respuesta, con 7 mil millones de parámetros se nota mucho.

Buscando otra cosa encontré una comparativa en [jdhodges.com](https://www.jdhodges.com/blog/local-llms-on-tool-calling-2026-pt1-local-lm/) (<https://www.jdhodges.com/blog/local-llms-on-tool-calling-2026-pt1-local-lm/>) donde probaron 13 modelos locales en 40 casos de test. Qwen3.5 4B sacó el mejor resultado: un 97,5% de acierto, por encima de modelos mucho más grandes. Intenté cargar el 4B pero mi equipo no lo soporta (LM Studio me daba un error raro de código de salida desbordado), así que me bajé al 0.8B de Unsloth.

El 0.8B pesa 1,21 GB en formato GGUF (un formato optimizado para ejecutar modelos en local) con cuantización Q8_0 (técnica que reduce el tamaño del modelo sin perder mucha calidad). Para mi herramienta con 3 parámetros de tipo enum (on/off/blink) va de sobra: el modelo sólo tiene que rellenar tres campos con valores de una lista cerrada.

Referencia del modelo: <https://lmstudio.ai/models/qwen3> y la guía de Unsloth: <https://unsloth.ai/docs/models/qwen3.5>

También vi otro artículo en [mayhemcode.com](https://www.mayhemcode.com) (<https://www.mayhemcode.com/2026/03/best-models-for-lm-studio-llama-4-qwen3.html>) que compara los mejores modelos para LM Studio y confirma que Qwen3.5 destaca en llamada a herramientas frente a Llama 4 y Mistral.

Configuración:

- Temperatura: 0.7 -- lo típico para que las respuestas no sean siempre idénticas

- Transmisión continua (stream): desactivada -- la respuesta llega de una vez, lo cual facilita parsear el JSON completo
- tool_choice: "required" cuando detecto que el mensaje tiene pinta de comando de luces. Esto obliga al modelo a usar una herramienta sí o sí, en vez de responder con texto tipo "claro, voy a encender la luz roja". Al principio no ponía esto y el modelo a veces no llamaba a la herramienta

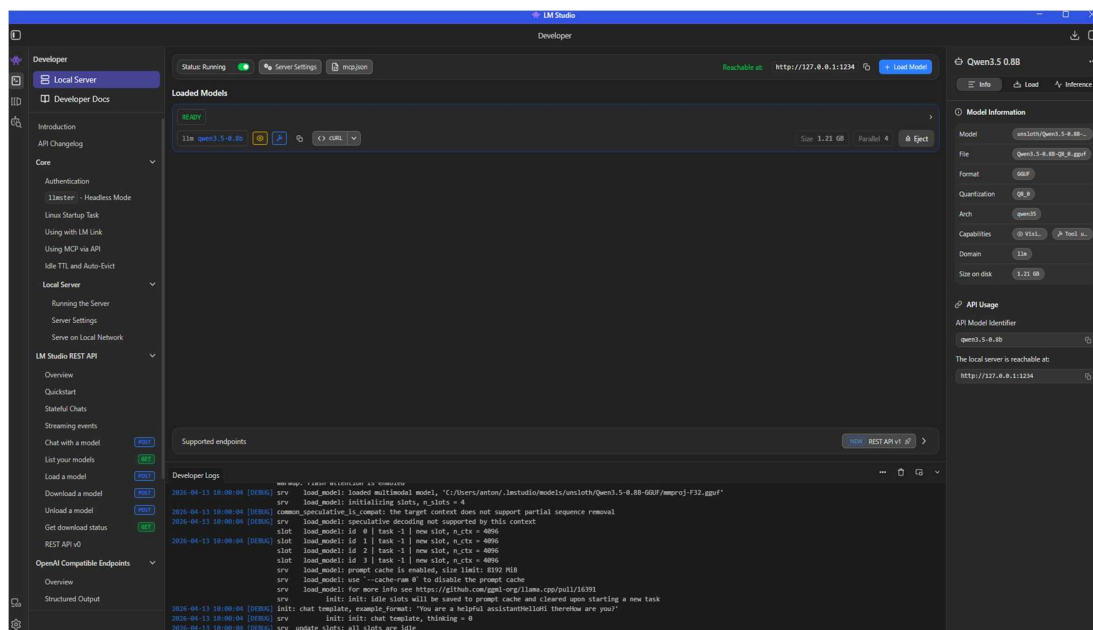


Figura 1: LM Studio con Qwen3.5 0.8B cargado y servidor activo

4.3 System prompt

El system prompt me costó varias vueltas. La primera versión era más suave y el modelo a veces responde con texto tipo “vale, ahora enciendo” en vez de llamar a la herramienta. Acabé poniéndole “**REGLA OBLIGATORIA**” en mayúsculas y “**NUNCA** respondas sólo con texto” en negrita mental, porque según la documentación de Qwen sobre function calling (https://qwen.readthedocs.io/en/latest/framework/function_call.html), los modelos pequeños necesitan que se lo dejes muy clarito para que lo hagan siempre.

Eres un asistente IoT que controla 3 luces físicas (verde, roja, azul).

Tienes UNA SOLA herramienta: `controlar_luces`, con estos parámetros:

- verde, roja, azul: estado de cada luz, uno de: "off", "on", "dim" (tenue), "blink" (parpadeo 400ms), "fast" (150ms), "slow" (1s), "fade" (sube y baja suave).
- modo_especial: opcional, uno de "ninguno" (por defecto), "semaforo", "ola", "morse". Si pones semaforo/ola/morse, los estados de verde/roja/azul se ignoran.
- texto_morse: sólo si modo_especial=morse, el texto a emitir.
- color_morse: sólo si modo_especial=morse, en qué LED emitir.

REGLA OBLIGATORIA: cuando el usuario pida tocar luces, SIEMPRE llama a `controlar_luces`. Mapeo de intenciones: "tenue/suave/bajito" -> dim;

"rápido" -> fast; "lento" -> slow; "respira/pulsa/fade" -> fade;

"parpadea" -> blink; "encendida/fija" -> on; "apagada" -> off.

Si dice "semáforo" -> modo_especial=semaforo.

Si dice "la ola" -> modo_especial=ola.

Si dice "morse" o "emite XXX en morse" -> modo_especial=morse, texto_morse=XXX.

Si pregunta algo no relacionado con luces, responde en texto normal.

Es más largo que el primero que tenía porque hay que explicarle las combinaciones y el mapeo de intenciones. Si digo “enciende la verde y pon la roja intermitente” tiene que deducir verde=on, roja=blink, azul=off. El bloque de mapeo de intenciones (tenue->dim, rápido->fast...) es clave porque el usuario nunca dice “dim” o “fade” en castellano natural, dice “tenue” o “respira”, y el modelo pequeño necesita que se lo dejes clarito.

5. Configuración de la herramienta (tool)

El ejemplo base del enunciado tenía 2 herramientas sin parámetros (`turn_on_light` y `turn_off_light`). Mi primera versión tenía 4 (una por color más apagar). Cuando me puse a combinar cosas (dos encendidas, una

parpadeando y otra fija, apagar sólo una...) me di cuenta de que con herramientas separadas no podía hacerlo. Así que lo refactoricé a una única herramienta con parámetros, que es un patrón mucho más potente de function calling.

La herramienta se llama controlar_luces y tiene 3 parámetros principales (verde, roja, azul), cada uno con 7 estados distintos: off, on, dim (tenue), blink (parpadeo 400ms), fast (rápido 150ms), slow (lento 1s) y fade (sube y baja suave). Además he añadido unos parámetros opcionales para activar modos automáticos como semáforo, ola o emitir texto en código morse, pero sigue siendo UNA única herramienta. Según la documentación de OpenAI sobre function calling

(<https://platform.openai.com/docs/guides/function-calling>), que es la API que LM Studio replica, puedes definir parámetros con enum para limitar los valores posibles. Eso ayuda mucho al modelo pequeño porque no tiene que inventar nada, sólo elegir de la lista.

Parámetros de controlar_luces:

Parámetro	Tipo	Valores posibles	Propósito
verde, roja, azul	string	off, on, dim, blink, fast, slow, fade	Estado de cada LED (obligatorio)
modo_especial	string	ninguno, semaforo, ola, morse	Si no es 'ninguno' se ignoran los 3 anteriores
texto_morse	string	letras A-Z, números, espacios (max 30)	Sólo si modo_especial=morse
color_morse	string	verde, roja, azul	Sólo si modo_especial=morse: por qué LED emitir

Ejemplos de lo que puede hacer:

El usuario dice	Argumentos que elige el modelo
"enciende la verde"	verde=on, roja=off, azul=off
"pon la azul tenue y la roja parpadeando rápido"	verde=off, roja=fast, azul=dim
"haz que la verde respire"	verde=fade, roja=off, azul=off
"verde parpadeo lento, roja rápido, azul fija"	verde=slow, roja=fast, azul=on
"activa el modo semáforo"	modo_especial=semaforo
"haz la ola"	modo_especial=ola
"emite SOS en morse por la roja"	modo_especial=morse, texto_morse=SOS, color_morse=roja
"apaga todo"	verde=off, roja=off, azul=off

El JSON de la herramienta (simplificado):

```
{
  "type": "function",
  "function": {
    "name": "controlar_luces",
    "description": "Controla las 3 luces. Si modo_especial no es 'ninguno' se activa el modo y se ignoran verde/roja/azul.",
    "parameters": {
      "type": "object",
```

```

    "properties": {
      "verde": { "type": "string", "enum":
["off","on","dim","blink","fast","slow","fade"] },
      "roja":  { "type": "string", "enum":
["off","on","dim","blink","fast","slow","fade"] },
      "azul":  { "type": "string", "enum":
["off","on","dim","blink","fast","slow","fade"] },
      "modo_especial": { "type": "string", "enum":
["ninguno","semaforo","ola","morse"] },
      "texto_morse":   { "type": "string" },
      "color_morse":   { "type": "string", "enum":
["verde","roja","azul"] }
    },
    "required": ["verde", "roja", "azul"]
  }
}

```

Para parsear la respuesta uso System.Text.Json, que ya viene con .NET y no necesita NuGet extra. El ejemplo del enunciado usaba Newtonsoft.Json, pero según Microsoft (<https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>) el que viene de serie es más rápido y es el recomendado. Ahora, además del nombre de la función, tengo que sacar los argumentos. Los argumentos vienen como un string JSON dentro del JSON de la respuesta (sí, JSON anidado), así que hay que parsearlos aparte:

```

// Saco los argumentos del tool_call
string argsJson = firstCall.GetProperty("function")
    .GetProperty("arguments").GetString() ?? "{}";
using var args = JsonDocument.Parse(argsJson);

// miro primero si el LLM ha elegido un modo especial
string modo = LeerCampo(args.RootElement, "modo_especial",

```

```

ModosEspeciales, "ninguno");
if (modo == "semaforo" || modo == "ola")    return new LmStudioResult {
Preset = modo };
if (modo == "morse") { /* saco texto_morse y color_morse */ }

// si no hay modo especial, leo los 3 LEDs normales
var ledStates = new Dictionary<string, string>
{
    ["verde"] = LeerEstado(args.RootElement, "verde"),
    ["roja"]  = LeerEstado(args.RootElement, "roja"),
    ["azul"]  = LeerEstado(args.RootElement, "azul")
};
// LeerEstado/LeerCampo validan contra la lista de estados permitidos
// y normalizan tildes, así si el modelo devuelve "encendido" o "ÓN" lo
mando a off

```

Uso TryGetProperty con valor por defecto "off" por si el modelo se deja algún parámetro sin rellenar. Con el enum cerrado es raro que pase, pero por si acaso no quiero que pete.

Sobre el tool_choice: cuando detecto que el usuario quiere tocar luces (busco palabras como “enciende”, “apaga”, “luz”, “intermitente”, “parpadea”...) fuerzo tool_choice = "required". Si parece pregunta (“¿qué luces puedes controlar?”) lo dejo en auto. Sin esto me pasó que el modelo contestaba “claro, voy a encender la verde” pero no llamaba a la herramienta y el LED no se encendía.

6. Aplicación .NET MAUI

6.1 Por qué MVVM y por qué CommunityToolkit

El enunciado pide MVVM. Hay varias formas de hacer MVVM en .NET MAUI, pero el ejemplo base tenía todo en el archivo de detrás de la vista (lo que se conoce como code-behind, el MainWindow.xaml.cs de turno), que es exactamente lo contrario de MVVM. Así que tocaba reorganizarlo.

He elegido CommunityToolkit.Mvvm (<https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>) porque es el que recomienda Microsoft, y sobre todo porque genera código automáticamente con source generators (generadores de código en tiempo de compilación) y te ahorra escribir INotifyPropertyChanged a mano, que es un rollo. Con los atributos [ObservableProperty] y [RelayCommand] escribes la mitad de código. Ya lo había usado en tareas anteriores y va bien.

Miré también Prism (<https://prismlibrary.com/>), que es un framework más completo, pero también más pesado. Para una app con dos pantallas no merecía la pena.

6.2 Estructura del proyecto

Carpeta	Archivos	Para qué sirve
Models/	ChatMessage.cs	Datos de cada mensaje: texto, si es del usuario, timestamp
Services/	LmStudioService.cs	Comunicación con LM Studio: envía mensajes con la herramienta

Carpeta	Archivos	Para qué sirve
		controlar_luces y parsea los argumentos JSON que devuelve
Services/	NodeRedService.cs	Envía por HTTP POST a Node-RED los estados de las 3 luces o activa un preset (semaforo)
ViewModels/	MainViewModel.cs	Lógica principal: chat, LEDs visuales, comandos por voz, botones manuales. Aquí está toda la lógica que en el ejemplo base estaba en el código trasero de la vista
ViewModels/	SettingsViewModel.cs	Ajustes de la app: URLs, modelo, broker (intermediario) MQTT. Consulta la API de LM Studio para listar los modelos disponibles
Views/	MainPage.xaml	Pantalla principal con chat, LEDs y barra de entrada
Views/	SettingsPage.xaml	Formulario de configuración
Converters/	BoolToAlignmentConverter.cs	Converters de XAML para alinear mensajes (derecha

Carpeta	Archivos	Para qué sirve
		para usuario, izquierda para bot) y color del micro
(raíz)	MauiProgram.cs	Inyección de dependencias: registra HttpClient, servicios, ViewModels y páginas

La inyección de dependencias va en MauiProgram.cs. Los servicios como Singleton (una sola instancia) para que no se pierda el historial al navegar entre páginas, y los ViewModels como Transient (se crea una instancia nueva cada vez que se necesitan). Al principio los tenía todos Transient y cada vez que iba a Ajustes y volvía me quedaba sin historial de chat. Esto de los tipos de vida lo aclara la docu de MAUI (<https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/dependency-injection>).

6.3 LmStudioService: cómo habla la app con el modelo

LmStudioService habla con LM Studio por HTTP. Al principio tenía guardado un historial de conversacion que se mandaba entero al modelo con cada petición, como se hace normalmente en un chat, para que recordase el contexto.

Pero probandolo me encontré con un problema serio: con un modelo tan pequeño (0.8B parámetros) el historial le liaba. Cuando la conversacion llevaba 10-15 mensajes, el modelo empezaba a mezclar respuestas anteriores con la nueva. Por ejemplo, si antes le habia dicho “activa el modo semaforo” y después le pedia “emite SOS en morse”, me respondia

“He activado el modo semaforo” otra vez, porque esa era la última acción del historial y se quedaba enganchado ahí. Probe con distintas frases, reinicie, force tool_choice, y aún así pasaba.

El motivo técnico es que los modelos pequeños tienen menos “capacidad de atención”: el mecanismo de attention del transformer funciona peor con contextos largos porque los pesos son limitados. Un modelo grande (como los de OpenAI o Qwen 4B) maneja fácilmente 20-30 mensajes sin perderse. Un modelo de 0.8B se atasca antes.

La solución que acabe poniendo fue mandar al LLM solo el system prompt y el mensaje actual del usuario. Nada de historial. Cada comando se interpreta en aislado y el modelo responde limpio. El trade-off: el modelo no puede hacer referencias a lo anterior (no entiende “pon la verde igual que estaba antes”), pero a cambio es mucho más fiable. Para este caso de uso (control de luces por comandos atómicos) es la decisión correcta.

El historial sigue existiendo en la app, pero solo para pintarlo en el chat de la UI: así el usuario ve sus mensajes anteriores y las respuestas, que es lo que espera de una interfaz de chat. Lo que cambia es que ese historial no se manda al LLM.

SendAsync monta la petición al endpoint /v1/chat/completions (documentación de la API en <https://lmstudio.ai/docs/api>). Antes de mandarla hace una detección: mira si el mensaje tiene pinta de comando (“enciende”, “apaga”, “intermitente”, “parpadea”...) o de pregunta (signo de interrogación, “qué”, “puedes”...). Si parece comando, fuerza tool_choice = “required” para que el modelo rellene los parámetros sí o sí. Si es pregunta, lo deja en auto:

```
// busco palabras clave: acciones, colores, modos y presets
bool tienePalabrasLuz =
    msgLower.Contains("encien") || msgLower.Contains("apaga") ||
    msgLower.Contains("prende") || msgLower.Contains("luz")    ||
    msgLower.Contains("led")   || msgLower.Contains("roj")    ||
    msgLower.Contains("verde") || msgLower.Contains("azul")   ||
```

```

msgLower.Contains("intermiten") || msgLower.Contains("blink") ||
msgLower.Contains("parpadea") || msgLower.Contains("fij") ||
msgLower.Contains("tenue") || msgLower.Contains("suave") ||
msgLower.Contains("rapido") || msgLower.Contains("lento") ||
msgLower.Contains("respira") || msgLower.Contains("pulsa") ||
msgLower.Contains("fade") || msgLower.Contains("semaforo") ||
msgLower.Contains("brillo") || msgLower.Contains(" ola") ||
msgLower.Contains("morse");

bool esPregunta = msgLower.Contains("?") || msgLower.Contains("qué")
|| msgLower.Contains("puedes") || msgLower.Contains("cómo");

bool esComandoLuz = tienePalabrasLuz && !esPregunta;

```

Esto me pasó: sin la detección de preguntas, cuando preguntaba “¿qué luces puedes controlar?” el modelo encendía una al azar en vez de contestarme. Porque yo le había puesto `tool_choice = required` para todo. La solución fue mirar antes si era pregunta y no forzarle nada.

Al parsear la respuesta tengo que sacar los argumentos. Como ya expliqué en la sección 2, vienen como JSON anidado. Monto un `Dictionary<string, string>` con las claves verde, roja, azul y si alguna falta pongo off por defecto.

6.4 MainViewModel: lo que pasa al darle a Enviar

MainViewModel conecta la interfaz con los servicios. Al pulsar Enviar se ejecuta `SendAsync`: añade tu mensaje al chat (la `ObservableCollection` que pinta los mensajes), llama a LM Studio, y según lo que venga hace una cosa u otra:

```

var result = await _lmStudioService.SendAsync(input);

if (result.IsToolCall)
    await ProcesarToolCall(result.LedStates!);
else
    Messages.Add(new ChatMessage { Text = result.TextResponse, IsUser =
false });

```

ProcesarToolCall recibe el diccionario con los 3 estados (por ejemplo verde=on, roja=blink, azul=off), actualiza los LEDs de la interfaz y manda el JSON a Node-RED:

```

private async Task ProcesarToolCall(Dictionary<string, string> ledStates)
{
    _currentStates = new Dictionary<string, string>(ledStates);
    SetLedsFromStates(ledStates);
    bool ok = await _nodeRedService.SendCommandAsync(ledStates);
    // ... mensaje al chat y haptic feedback
}

```

SetLedsFromStates pone cada LED en su color con opacidad alta (encendido) o gris con opacidad baja (apagado). Si algún LED está en blink, arranco un DispatcherTimer que lo alterna cada 400ms entre color vivo y gris, para que el efecto visual de parpadeo se vea en la app. El timer lo paro y lo vuelvo a crear cada vez que llega un estado nuevo, si no se acumulan y parpadea a lo loco.

Ahora pueden estar varios LEDs encendidos a la vez, o uno fijo y otro parpadeando, o los tres intermitentes. Todo lo decide el modelo con los 3 parámetros.

6.5 Comandos por voz

Esto no lo pide el enunciado pero me apetecía meterlo. Uso `CommunityToolkit.Maui` (<https://learn.microsoft.com/en-us/dotnet/communitytoolkit/maui/>) porque ya tenía `CommunityToolkit.Mvvm` del mismo paquete y la integración es inmediata. No hace falta claves de API ni depender de nada externo.

Miré `Azure Speech Services` pero te obliga a tener cuenta y clave, `Vosk` va offline pero engancharlo a MAUI era un lío, y la API nativa de Windows sólo vale en Windows. El del `CommunityToolkit` lo resuelve todo sin dramas.

Lo fuerzo a español (es-ES) porque si no a veces me detectaba palabras en inglés y la líaba. Mientras hablas va mostrando lo que reconoce, y al terminar se manda al LLM sólo:

```
var options = new SpeechToTextOptions
{
    Culture = CultureInfo.GetCultureInfo("es-ES"),
    ShouldReportPartialResults = true
};
await speechToText.StartListenAsync(options);
```

Los permisos de micro me dieron guerra. En Windows la primera vez pide permiso y ya. En Android tienes que añadir `RECORD_AUDIO` al `AndroidManifest.xml` o la app petará al pulsar el micro. Lo explica la docu de permisos de MAUI (<https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/appmodel/permissions>).

6.6 Interfaz de usuario

La interfaz es oscura, tipo panel de control IoT. Me gusta que la pinta acompañe a lo que hace la app. A la izquierda el chat con estilo terminal (fondo casi negro, texto claro), a la derecha los tres círculos que se

iluminan con un halo (la Shadow de MAUI) cuando se enciende el LED. Arriba un panel con información del sistema: modelo cargado, broker y topic. Abajo un log con la última acción.

La barra de entrada tiene tres cosas: el campo de texto, un botón de micro para la voz, y Enviar. Los mensajes del usuario salen en azul a la derecha y los del bot en gris a la izquierda. Lo de la alineación se hace con un Converter de XAML, el BoolToAlignmentConverter, que mira el IsUser del mensaje y devuelve Start o End.

Debajo tengo cuatro botones manuales (verde, roja, azul y apagar todas) para poder probar el sistema sin depender del LLM. Los tres de colores funcionan como un ciclo de 3 estados: primer clic enciende, segundo clic pone en intermitente, tercer clic apaga. Cada botón sólo afecta a su color, así que puedo combinar estados a mano (por ejemplo verde encendida + roja parpadeando + azul apagada) dando clics independientes. El botón Apagar manda las tres a off de golpe. Esto me salvó la vida al depurar: podía comprobar el enlace app-Node-RED-ESP32 sin tener que lanzar LM Studio cada vez.

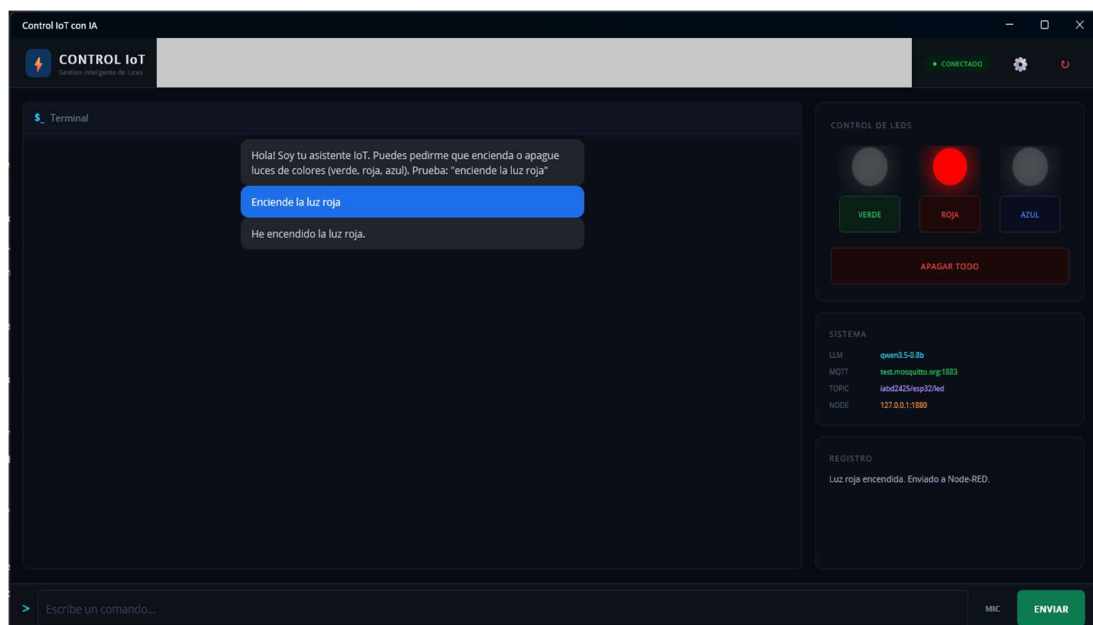


Figura 2: App MAUI con la luz roja encendida tras comando por chat

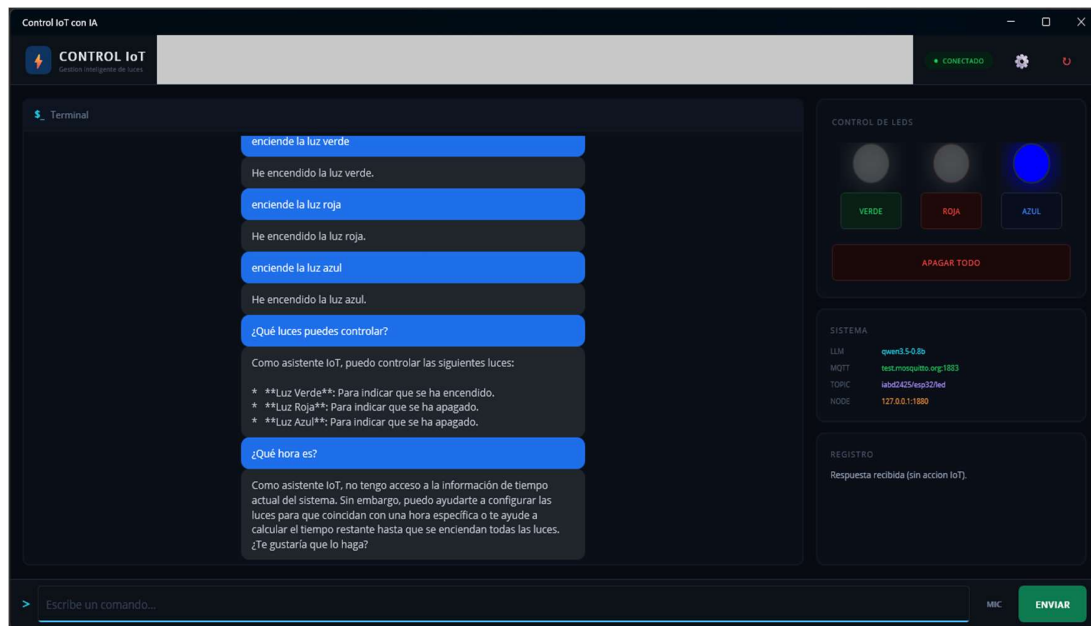


Figura 3: Conversación extendida con múltiples comandos de luces

7. Node-RED

7.1 Por qué Node-RED

Node-RED lo pone el enunciado como intermediario entre la app y MQTT. Podría publicar directamente desde la app con MQTTnet, pero así tengo una capa intermedia que puedo tocar sin recompilar la app. Si mañana quiero añadir un log, un filtro o mandar a dos brokers, lo hago en Node-RED arrastrando un nodo y ya. Y como expone un endpoint HTTP, lo puedo probar con curl sin levantar nada de .NET.

Node-RED es de la fundación OpenJS (<https://nodered.org/>). Se instala con npm y se configura en el navegador arrastrando nodos, que para un visual como yo va bien.

7.2 El flujo

El flujo tiene 5 nodos y es bastante directo:

Nodo	Tipo	Función
POST /encender	HTTP IN	Recibe las peticiones POST de la app MAUI
Preparar JSON	Function	Convierte el objeto JSON a string para publicarlo en MQTT tal cual
Publicar MQTT	MQTT OUT	Publica en test.mosquitto.org:1883, topic iabd2425/esp32/led
Respuesta HTTP	HTTP Response	Devuelve 200 a la app para que sepa que llegó
Debug (depuración)	Debug	Muestra el comando en la consola de Node-RED

El nodo Function:

```
// convierte el objeto a string JSON para que MQTT lo publique tal cual
// el ESP32 espera: {"verde":"on","roja":"off","azul":"blink"}
msg.payload = JSON.stringify(msg.payload);
return msg;
```

7.3 Broker MQTT

Uso el broker público test.mosquitto.org en el puerto 1883, sin usuario ni contraseña. Es el que sale en el enunciado y el que todo el mundo usa para trastear (<https://test.mosquitto.org/>). Ojo que se cae de vez en cuando,

sobre todo los fines de semana. Como respaldo tengo apuntado broker.hivemq.com por si acaso.

El topic es iabd2425/esp32/led. Da igual el nombre, lo importante es que la app y la ESP32 estén hablando en el mismo, que si no no se entera nadie.

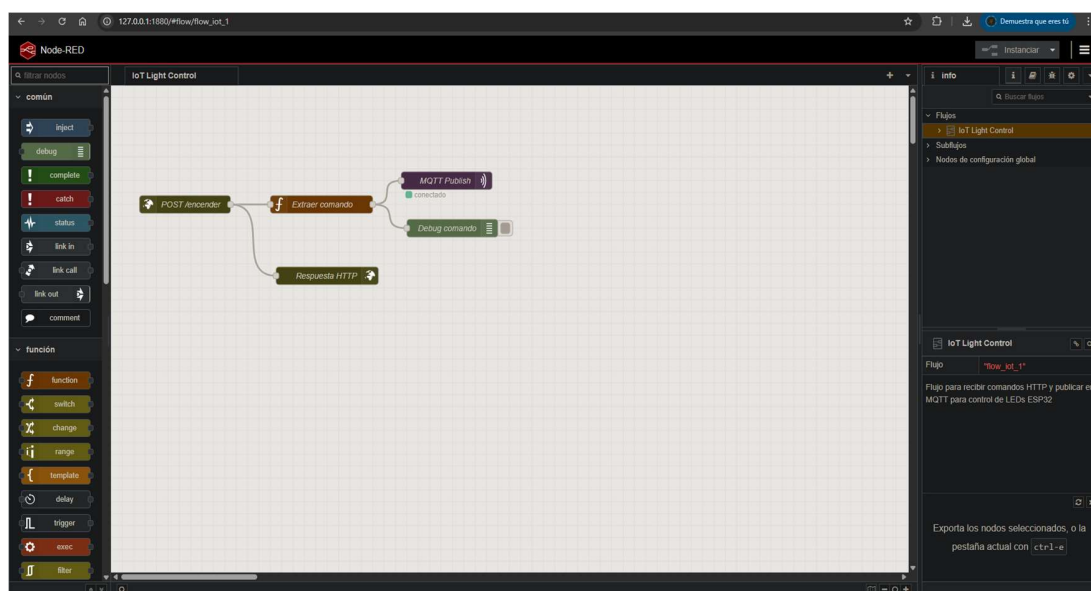


Figura 4: Flujo Node-RED con los 5 nodos (HTTP IN, Function, MQTT OUT, HTTP Response, Debug)

8. ESP32 y conexasión

8.1 Hardware y por qué PlatformIO

Uso una ESP32 DevKit V1 real, no simulación, enchufada por USB. El firmware lo escribo con PlatformIO (<https://platformio.org/>) dentro de VS Code, con el framework Arduino.

Se podía hacer con el Arduino IDE, pero PlatformIO me gusta más: las librerías las declaras en el platformio.ini y se descargan solas, tiene autocompletado de verdad, y ya trabajo en VS Code con todo lo demás, así que no me cambio de ventana. La docu oficial para ESP32 está aquí: <https://docs.platformio.org/en/latest/boards/espressif32/esp32dev.html>

Además de los 3 LEDs que pide el enunciado he metido una pantalla OLED SSD1306 de 0,96 pulgadas por I2C, que es la mejora opcional que mencionan (“incluir una pantalla LCD que muestre el mensaje recibido”). Preferí una OLED a una LCD porque por I2C (un protocolo de comunicación serie que usa sólo dos cables) sólo necesitas dos pines (SDA y SCL) y las LCD paralelas te comen seis o más. Además la librería Adafruit_SSD1306 es muy sencilla y hay mil ejemplos.

8.2 Conexionado

Componente	Pin GPIO	Conexión
LED Rojo	GPIO 15	GPIO 15 -> R 220 Ohm -> Ánodo -> Cátodo -> GND
LED Verde	GPIO 2	GPIO 2 -> R 220 Ohm -> Ánodo -> Cátodo -> GND
LED Azul	GPIO 4	GPIO 4 -> R 220 Ohm -> Ánodo -> Cátodo -> GND
OLED SDA	GPIO 21	I2C datos (pin por defecto de la ESP32)
OLED SCL	GPIO 22	I2C reloj (pin por defecto de la ESP32)
OLED VCC	VIN (5V)	Alimentación
OLED GND	GND	Tierra común

Los GPIO los elegí esquivando los que dan problemas en el arranque de la ESP32 (hay algunos que si están en HIGH durante el boot la placa no arranca, es un clásico). El GPIO 2 ya trae un LED integrado en la placa, así que cuando se enciende el verde también parpadea el de la placa, que

queda hasta simpático. Los pines 21 y 22 son los I2C por defecto según el pinout oficial de la DevKit V1 (<https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html>).

Las resistencias son de 220 ohmios, el valor típico para LEDs a 3,3V. Si echas números: $R = (V_{\text{gpio}} - V_{\text{led}}) / I_{\text{led}} = (3,3 - 2,0) / 0,005 = 260 \text{ ohm}$. 220 se queda un pelín por debajo pero está dentro del rango seguro y es lo que tenía en el kit.

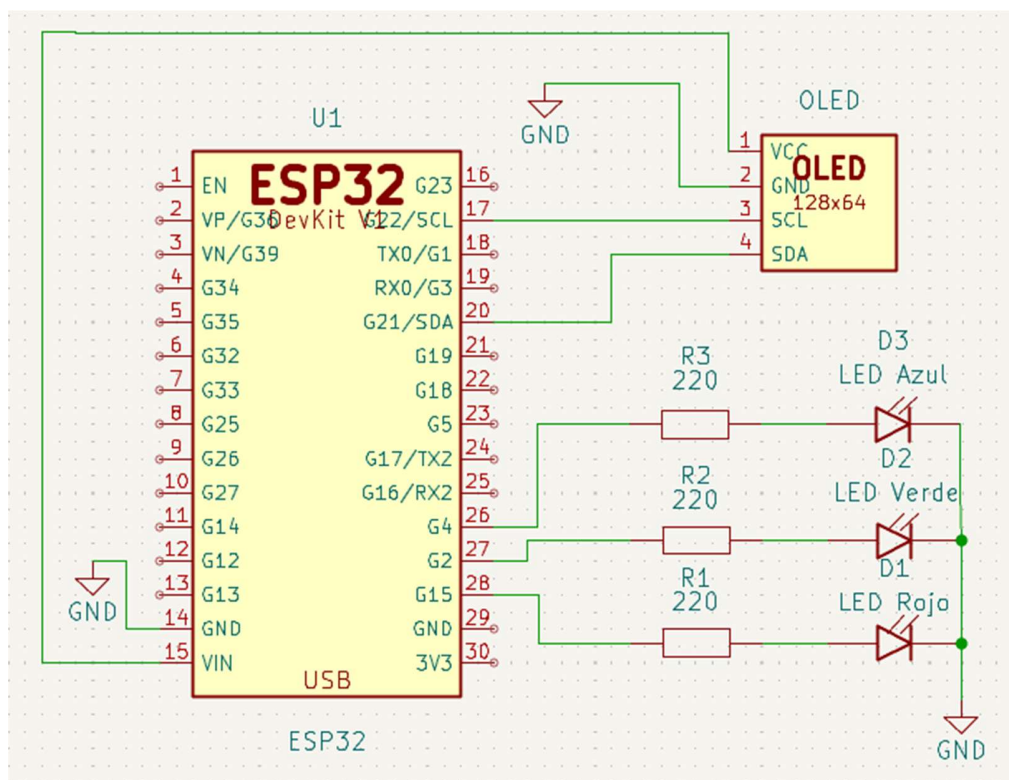


Figura 5: Esquema de conexionado ESP32 con 3 LEDs, resistencias y pantalla OLED (KiCad)

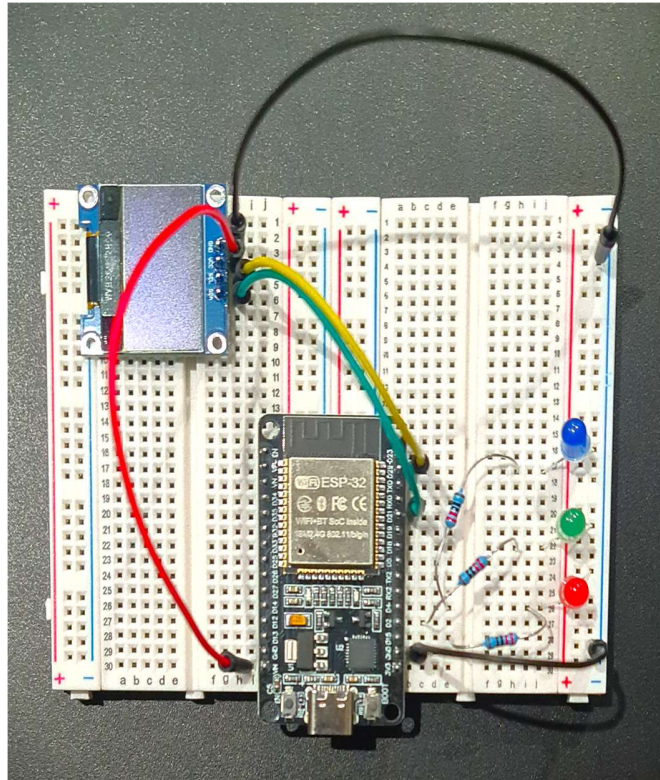


Figura 6: Montaje real en protoboard — ESP32, OLED SSD1306 y 3 LEDs con resistencias

8.3 Setup: WiFi, MQTT y OLED

El `setup()` hace las cosas en orden. Primero `Wire.begin(SDA, SCL)` arranca el bus I2C en los pines 21 y 22, y `display.begin()` enciende la OLED en la dirección `0x3C`. Esa dirección es la que traen casi todos los módulos SSD1306 de 0.96 pulgadas. Si por lo que sea la pantalla no arranca, saco un aviso por Serial y sigo sin ella; lo importante es que los LEDs sigan funcionando aunque la OLED esté rota o mal conectada.

Después configuro los pines de los LEDs como OUTPUT y los pongo a LOW. Así al arrancar siempre está todo apagado, aunque la placa haya quedado en un estado raro después de un reset o un error.

Luego viene el WiFi. `WiFi.begin(ssid, password)` y a esperar. Mientras no tiene IP, se queda en un `while` mostrando puntos en la OLED para que se vea que está intentando conectar. Cuando ya está, configura el cliente

MQTT con setServer y registra la función callback, que es la que se ejecutará cada vez que llegue un mensaje al topic al que está suscrito.

8.4 Reconexión MQTT

El broker público de mosquitto se cae de vez en cuando, o a veces lo que se corta es el WiFi. En el loop principal miro si el cliente sigue conectado y si no, llamo a reconnect(). Lo clave es que la reconexión es NO

BLOQUEANTE: en vez de un while con delay(), uso millis() para reintentar una sola vez cada 3 segundos y vuelvo al loop. Así los LEDs en blink y el OLED siguen respondiendo mientras busca broker. El client ID lo genero aleatorio con formato ESP32_XXXX (sprintf sobre buffer en pila, no String, para no fragmentar el heap del ESP32).

```
unsigned long ultimoReintento = 0;

void reconnect() {
    if (client.connected()) return;
    if (millis() - ultimoReintento < 3000) return; // reintento cada 3s
    ultimoReintento = millis();

    char cid[20];
    sprintf(cid, sizeof(cid), "ESP32_%04X", (unsigned
int)random(0xffff));

    if (client.connect(cid)) {
        client.subscribe(mqttTopic);
    }
    // si falla, vuelvo al loop y reintentara en 3s
}
```

8.5 Callback MQTT: cuando llega un comando

La función callback salta cada vez que llega un mensaje al topic. Antes comparaba strings sueltos (“encender_roja”, “apagar”...), ahora llega un JSON con los 3 estados. Lo parseo con ArduinoJson (<https://arduinojson.org/v7/tutorial/deserialization/>), que es la librería de JSON más usada en Arduino y ESP32.

El payload de MQTT llega como un array de bytes sin terminador \0, así que primero lo copio a un buffer con terminador. Después deserializeJson lo parsea y saco cada estado. Si falta algún campo pongo off por defecto:

```
void callback(char* topic, byte* payload, unsigned int length) {
    char buf[256];
    memcpy(buf, payload, length < 255 ? length : 255);
    buf[length < 255 ? length : 255] = '\0';

    JsonDocument doc; // ArduinoJson v7, sin tamaño fijo
    DeserializationError err = deserializeJson(doc, buf);
    if (err) return; // JSON roto, ignorar

    // tres ramas posibles según lo que traiga el JSON
    if (!doc["morse"].isNull()) {
        // {"morse":"SOS","color":"roja"} -> emitir texto en morse
        preset = "morse";
        prepararMorse(doc["morse"]);
    } else if (!doc["preset"].isNull()) {
        // {"preset":"semaforo"} o "ola" -> secuencia automática
        preset = (const char*)doc["preset"];
    } else {
        // {"verde":"on","roja":"off","azul":"dim"} -> control por LED
        preset = "ninguno";
        estadoVerde = parseEstado(doc["verde"] | "off");
    }
}
```

```

        estadoRojo = parseEstado(doc["roja"] | "off");
        estadoAzul = parseEstado(doc["azul"] | "off");
    }

    aplicarLeds();
    oledEstado();

    // ACK de vuelta: publico un JSON en iabd2425/esp32/ack para que
    // la app sepa que el ESP32 llego a procesar el comando
    client.publish(ackTopic, "{\"ok\":true}");
}

```

Cada LED tiene 7 estados posibles, guardados en un enum. Así me ahorro magic numbers y el código se lee mejor.

Valor JSON	Enum interno	Acción en el LED
"off"	OFF	Apagado (PWM = 0)
"on"	ON	Brillo máximo (PWM = 255)
"dim"	DIM	Brillo tenue (PWM = 20, casi vela)
"blink"	BLINK	Parpadea a 400 ms entre encendido y apagado
"fast"	FAST	Parpadea rápido, a 150 ms
"slow"	SLOW	Parpadea lento, a 1 segundo
"fade"	FADE	Sube y baja suave tipo respiración (PWM sinusoide)

8.6 PWM, parpadeo sin delay, preset semáforo y OLED

Los estados "on", "off" y los de parpadeo podrían hacerse con digitalWrite, pero "dim" y "fade" necesitan brillo intermedio. Para eso uso el módulo LEDC del ESP32 (PWM por hardware), configurado a 5 kHz y 8 bits de resolución. PWM significa Pulse Width Modulation: la placa enciende y apaga el LED muy rápido, y cambiando el porcentaje de tiempo encendido (el "ancho de pulso") se percibe como más o menos brillo. A 5 kHz el parpadeo es invisible para el ojo. Ref:

<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/ledc.html>

Configuro un canal PWM por LED:

```
ledcSetup(CANAL_ROJO, 5000, 8);    // 5 kHz, 0..255
ledcSetup(CANAL_VERDE, 5000, 8);
ledcSetup(CANAL_AZUL, 5000, 8);
ledcAttachPin(LED_ROJO, CANAL_ROJO);
ledcAttachPin(LED_VERDE, CANAL_VERDE);
ledcAttachPin(LED_AZUL, CANAL_AZUL);
```

El parpadeo (blink/fast/slow) lo hago con millis() en el loop(), sin delay(). Cada LED tiene su propio timer, así pueden ir a ritmos distintos (uno rápido, otro lento, otro fijo). La técnica "blink without delay" está explicada en los ejemplos oficiales de Arduino (<https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/>).

Para el modo fade cálculo una curva suave tipo respiración con una función seno, que va de 0 a 255 y vuelve con periodo de 2 segundos:

```
float fase = 2 * PI * (millis() % 2000) / 2000.0;
int fadeLvl = (int)((sin(fase) * 0.5 + 0.5) * 255);
ledcWrite(CANAL_VERDE, fadeLvl); // brillo continuo cambiando
```

El preset "semaforo" es una secuencia que se autogestiona en el loop: verde 3 segundos, ámbar (verde+rojo a la vez) 1 segundo, rojo 3 segundos, y vuelta a empezar. Se activa desde la única herramienta poniendo `modo_especial=semaforo`, y se desactiva poniendo `modo_especial=ninguno`. Cualquier comando normal de LEDs también lo interrumpe automáticamente.

El preset "ola" es parecido pero más simple: enciende los 3 LEDs por turnos (verde -> roja -> azul -> verde...) cada 500 ms, como cuando la grada de un estadio hace la ola. El usuario sólo tiene que decir "haz la ola" y el modelo rellena `modo_especial=ola`.

Modo "morse": emite un texto en código morse internacional por el LED que diga el usuario. El usuario dice por ejemplo "emite SOS en morse por la roja" y el modelo rellena `modo_especial=morse`, `texto_morse=SOS`, `color_morse=roja`. El ESP32 tiene una tabla con los símbolos morse de cada letra A-Z y dígitos 0-9, y los va emitiendo respetando los tiempos estándar: un punto dura 350 ms y la raya el triple, 1050 ms, para que en el LED se note claramente la diferencia

(https://es.wikipedia.org/wiki/C%C3%B3digo_morse). Entre los símbolos de una misma letra meto un gap de 1 unidad OFF (si no, las tres puntos de la S se verían como un solo parpadeo continuo). Entre letras uso 3 unidades OFF y entre palabras 7. Cuando la cadena termina, el firmware la vuelve a emitir desde el principio en bucle, así el morse se queda sonando hasta que el usuario mande apagar. Mientras emite, la OLED muestra en letras grandes los puntos y rayas que están saliendo (la implementación interna usa un carácter 'g' para marcar los gaps de una misma letra, pero la OLED los filtra para que solo se vean puntos, rayas y espacios).

La OLED está rediseñada para leerse de un vistazo. En la franja amarilla (filas superiores de la SSD1306 dual-color) va la cabecera, y en la franja azul van los 3 LEDs con su estado en castellano: "VERDE encendida", "ROJA parpadea", "AZUL tenue". Un círculo sólido indica que el LED está activo en cualquier modo; vacío que está apagado. Al recibir cada comando se

muestra un pantallazo breve "OK - Comando recibido" como feedback visual.

ACK de vuelta a la app (acuse de recibo): además de aplicar el comando, el ESP32 publica en el topic `iabd2425/esp32/ack` un JSON con los estados aplicados. Así cualquier cliente puede verificar que la placa llegó a procesar el mensaje. Ejemplo de payload ACK:

```
{"ok":true,"verde":2,"roja":3,"azul":0,"preset":"ninguno"}
```

Para MQTT uso PubSubClient (<https://pubsubclient.knolleary.net/api>). Hay otra librería AsyncMqttClient que no bloquea el loop, pero para comandos esporádicos como los de la demo PubSubClient va de sobra.

9. Servidor MCP

Además de la app MAUI he montado un servidor MCP en un proyecto aparte (LucesIoT.MCP). La idea es que cualquier cliente que hable Model Context Protocol, como Claude Desktop, pueda usar la misma herramienta `controlar_luces` sin tener que abrir la aplicación de escritorio. El servidor reusa la misma lógica de Node-RED mediante HTTP, así que todo el flujo hacia el ESP32 no cambia.

MCP es un estándar abierto de Anthropic (<https://modelcontextprotocol.io/>). La idea es que cualquier cliente que hable MCP pueda usar el mismo servidor, sin acoplarse a una app concreta. Está bastante de moda ahora mismo.

El servidor va en un proyecto .NET aparte (LucesIoT.MCP) con el paquete `ModelContextProtocol` de NuGet y transporte STDIO (entrada/salida estándar por consola), que es el más simple para un servidor local.

Herramienta MCP	Parámetros	Descripción
ControlarLuces	verde, roja, azul + modo_especial, texto_morse, color_morse	Todo en una: control de los 3 LEDs con 7 estados, o activa semáforo/ola/morse según modo_especial
EstadoSistema	ninguno	Devuelve info del estado actual, topic MQTT y modos disponibles

```
[McpServerTool, Description(
    "Controla las 3 luces. Estados por luz: off, on, dim, blink, fast,
    slow, fade. " +
    "Si modo_especial no es 'ninguno' se ignoran verde/roja/azul y se
    activa el modo.")]
public async Task<string> ControlarLuces(
    [Description("Estado de la luz verde")] string verde = "off",
    [Description("Estado de la luz roja")]  string roja  = "off",
    [Description("Estado de la luz azul")]  string azul  = "off",
    [Description("Modo: ninguno, semaforo, ola, morse")] string
modo_especial = "ninguno",
    [Description("Texto a emitir si modo=morse")]      string
texto_morse   = "",
    [Description("LED del morse: verde, roja, azul")]  string
color_morse   = "verde")
{
    // segun el modo elegido llama a una u otra ruta del servicio interno
    if (modo_especial == "semaforo" || modo_especial == "ola")
        return await _nodeRedService.SendPresetAsync(modo_especial) ? ...
    : "error";
```

```
    if (modo_especial == "morse" &&  
    !string.IsNullOrEmpty(texto_morse))  
        return await _nodeRedService.SendMorseAsync(texto_morse,  
color_morse) ? ... : "error";  
        return await _nodeRedService.SendLedStatesAsync(verde, roja, azul) ?  
... : "error";  
    }
```

Por dentro hace lo mismo que la app MAUI: decide si es modo normal, preset o morse, y manda el POST JSON correspondiente a Node-RED. La única diferencia es de dónde viene la orden: del chat de la app o de un cliente MCP como Claude Desktop. He mantenido la misma herramienta única con los mismos parámetros para que sea consistente entre los dos sitios.

10. Instalación y puesta en marcha

Son varias piezas que tienen que arrancar en el orden correcto: LM Studio, Node-RED, la ESP32 y la app. Si saltas un paso o los enciendes desordenados te puedes volver loco buscando por qué no responde. Lo explico en orden para que sea fácil de seguir.

Para ahorrarme los pasos cada vez he incluido en la raíz del proyecto un script `ARRANCAR_TODO.bat` que levanta el servidor de LM Studio con el modelo cargado, arranca Node-RED y lanza la app MAUI en cadena. Sólo hay que darle doble clic. El script detecta si tengo instalado el CLI de LM Studio (`lms`) y lo usa; si no, avisa para abrir LM Studio a mano. Igual con Node-RED: si no lo encuentra, lo intenta con `npx`. Así la demo arranca sin tener que abrir tres programas por separado.

10.1 Requisitos previos

Software necesario:

- Windows 10/11 (también compila para Android/iOS pero yo lo he probado en Windows)
- .NET 9.0 SDK -- descarga: <https://dotnet.microsoft.com/download>
- Node.js y npm -- descarga: <https://nodejs.org> (la versión LTS)
- LM Studio -- descarga: <https://lmstudio.ai>
- Un IDE para la app MAUI: Visual Studio 2022 (17.8+) con carga de trabajo ".NET Multi-platform App UI development", VS Code con C# Dev Kit, o terminal con dotnet CLI
- PlatformIO -- se instala como extensión de VS Code (buscar "PlatformIO IDE" en la tienda de extensiones)

Hardware para la parte IoT:

- ESP32 DevKit V1
- 3 LEDs (rojo, verde, azul) con 3 resistencias de 220 ohmios
- Pantalla OLED SSD1306 0.96" I2C (opcional)
- Cables dupont y protoboard
- Cable USB para la ESP32

10.2 Instalar y arrancar LM Studio

1. Descargar LM Studio desde <https://lmstudio.ai> e instalarlo.
2. Abrir LM Studio. En la barra de búsqueda de modelos, buscar "unsloth/qwen3.5" y descargar Qwen3.5-0.8B-GGUF (el de unsloth, 1.21 GB). Si tu equipo tiene más RAM/GPU, puedes probar el 4B que es mejor.
3. Ir a la pestaña Developer > Local Server.
4. Pulsar "+ Load Model" y seleccionar el Qwen3.5 0.8B que acabas de descargar.

5. El servidor arranca automáticamente. Verificar que funciona abriendo en el navegador:

```
http://127.0.0.1:1234/v1/models
```

Debería devolver un JSON con el modelo. Si sale error, comprobar que no haya otro proceso en el puerto 1234.

10.3 Instalar y arrancar Node-RED

Node-RED necesita Node.js. Si no lo tienes, descárgalo de <https://nodejs.org> (LTS).

Instalar Node-RED de forma global:

```
npm install -g node-red
```

Arrancarlo:

```
node-red
```

Esto levanta Node-RED en <http://localhost:1880>. Abrirlo en el navegador.

Importar el flujo del proyecto:

1. En Node-RED, ir al menú (tres rayas arriba a la derecha) > Import
2. Seleccionar el archivo NodeRED/flows.json de la carpeta del proyecto
3. Pulsar Import
4. Pulsar Deploy (desplegar, botón rojo arriba a la derecha)

Para comprobar que funciona, abrir una terminal y hacer:

```
curl -X POST http://localhost:1880/encender -H "Content-Type: application/json" -d '{"verde":"off","roja":"on","azul":"off"}'
```

O en PowerShell:

```
Invoke-RestMethod -Uri http://localhost:1880/encender -Method Post -  
ContentType "application/json" -Body  
'{"verde":"off","roja":"on","azul":"off"}'
```

Si en la pestaña Debug (depuración) de Node-RED aparece el JSON con los 3 estados y después el LED rojo del ESP32 se enciende, está funcionando bien.

10.4 Cargar firmware en la ESP32

1. Abrir VS Code e instalar la extensión PlatformIO IDE (si no la tienes, buscarla en la tienda de extensiones).
2. Abrir la carpeta ESP32/ del proyecto: File > Open Folder > seleccionar la carpeta ESP32.
3. Editar src/main.cpp y poner tu WiFi:

```
const char* ssid = "TU_WIFI";  
const char* password = "TU_PASSWORD";
```

1. Conectar la ESP32 por USB al ordenador.
2. En la barra azul inferior de VS Code (la de PlatformIO), pulsar el icono de flecha para subir el firmware. La primera vez tarda más porque PlatformIO descarga las librerías (PubSubClient, Adafruit SSD1306, Adafruit GFX).
3. Abrir el Serial Monitor (monitor serie, icono del enchufe en la barra de PlatformIO, velocidad 115200). Debería mostrar:

```
Conectando a WiFi...  
WiFi conectado! IP: 192.168.1.xxx  
Conectando a MQTT...  
Conectado a MQTT. Suscrito a: iabd2425/esp32/led
```

Si no conecta al WiFi, verificar SSID y contraseña. Si no conecta a MQTT, puede ser que test.mosquitto.org esté caído (pasa a veces) -- en ese caso cambiar al broker de respaldo broker.hivemq.com.

10.5 Compilar y ejecutar la app MAUI

Opción A -- Visual Studio 2022 (lo más sencillo):

1. Abrir LucesIoT.slnx con Visual Studio 2022.
2. En el desplegable de target, seleccionar "Windows Machine".
3. Pulsar F5 o el botón de play verde.

Opción B -- Terminal con dotnet CLI (sin Visual Studio):

```
cd LucesIoT.MAUI
dotnet restore
dotnet build -f net9.0-windows10.0.19041.0
dotnet run -f net9.0-windows10.0.19041.0
```

Opción C -- VS Code:

Abrir la carpeta del proyecto, instalar la extensión C# Dev Kit, y usar Ctrl+F5.

Paquetes NuGet que usa la app (se restauran automáticamente):

- CommunityToolkit.Mvvm 8.4.2 -- patrón MVVM
- CommunityToolkit.Maui 11.2.0 -- reconocimiento de voz (SpeechToText)
- Microsoft.Maui.Controls -- el framework de MAUI (viene con el SDK)

10.6 Orden de arranque

Para que todo funcione de extremo a extremo:

1. LM Studio -- cargar el modelo y verificar que el servidor está en "Running"
2. Node-RED -- arrancar y desplegar el flujo
3. ESP32 -- conectar por USB y verificar en el monitor serie que conecta al WiFi y MQTT
4. App MAUI -- ejecutar con F5 o dotnet run

Si todo está correcto, al escribir 'enciende la luz roja' en la app: el chat muestra el mensaje, el LED visual de la app se pone rojo, en Node-RED aparece el mensaje de depuración, y en la ESP32 se enciende el LED físico rojo.

11. Conclusiones

Mirando atrás, lo importante me ha salido: que un modelo corriendo en mi propio PC entienda castellano normal (“enciende la verde y pon la roja parpadeando rápido”) y acabe moviendo unos LEDs físicos en la ESP32 al otro lado de la red. La primera vez que la placa respondió en dos segundos me reí solo. La cadena entera (app MAUI → LM Studio → Node-RED → MQTT → ESP32) va estable, y al montarla he entendido por fin cómo encajan las piezas que antes veía sueltas.

No todo fue fácil. Donde más me atasqué fue eligiendo modelo: probé tamaños y cuantizaciones a ciegas hasta dar con uno que me cupiera en la GPU y además llamase a la tool de verdad. El system prompt también me hizo sudar, porque el modelo contestaba “vale, ahora enciendo” en vez de invocar la herramienta; no se quitó hasta que metí la regla en mayúsculas y forcé tool_choice=“required”. Y en la ESP32 me comió un par de horas los delay() bloqueantes al meter varios parpadeos a la vez; lo reescribí con millis() y cada LED con su propio ritmo.

Me llevo varias cosas. La principal, que diseñar bien la tool (parámetros, enum, descripción) importa casi más que elegir modelo: con una tool bien pensada hasta un modelo pequeño acierta. También que partir el sistema en capas (MVVM en la app, Node-RED como pasarela, ESP32 solo ejecutando) te salva cuando algo peta, porque vas probando trozo a trozo. Y, ya al final, que MQTT para esto es un regalo: con cuatro líneas tienes comunicación bidireccional y desacoplada.

Cosas que dejo pendientes: meter TLS y autenticación al broker MQTT (ahora mismo está todo abierto) y sacar SSID/password a un fichero de

config en vez de tenerlos dentro del código. Si lo retomara, añadiría más dispositivos (otra ESP32, un sensor de temperatura, un relé) y un inventario dinámico para que el LLM supiera qué hay conectado en cada momento. Si acabo cambiando de equipo probaré con un modelo más grande para ver cuánto mejora con frases ambiguas. Y ya puestos, estaría guapo guardar logs persistentes de lo que se va ejecutando para revisar después qué hizo el sistema.

12. Bibliografía y referencias

Todas las fuentes consultadas durante el desarrollo de este proyecto, organizadas por tema:

12.1 Modelo LLM y function calling

- LM Studio -- aplicación para ejecutar LLMs en local:
<https://lmstudio.ai>
- Qwen3.5 en LM Studio: <https://lmstudio.ai/models/qwen3>
- Unsloth Qwen3.5-0.8B-GGUF (modelo utilizado):
<https://huggingface.co/unsloth/Qwen3.5-0.8B-GGUF>
- Comparativa de modelos locales en tool calling (jdhodges.com, 13 modelos, 40 tests): <https://www.jdhodges.com/blog/local-llms-on-tool-calling-2026-pt1-local-lm/>
- Mejores modelos para LM Studio (mayhemcode.com):
<https://www.mayhemcode.com/2026/03/best-models-for-lm-studio-llama-4-qwen3.html>
- Documentación de Qwen sobre function calling:
https://qwen.readthedocs.io/en/latest/framework/function_call.html
- Guía de function calling de OpenAI (API que LM Studio replica):
<https://platform.openai.com/docs/guides/function-calling>

12.2 .NET MAUI y MVVM

- CommunityToolkit.Mvvm (Microsoft): <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/mvvm/>
- CommunityToolkit.Maui (SpeechToText): <https://learn.microsoft.com/en-us/dotnet/communitytoolkit/maui/>
- Inyección de dependencias en MAUI: <https://learn.microsoft.com/en-us/dotnet/maui/fundamentals/dependency-injection>
- Permisos en MAUI: <https://learn.microsoft.com/en-us/dotnet/maui/platform-integration/appmodel/permissions>
- System.Text.Json (Microsoft): <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview>

12.3 Node-RED y MQTT

- Node-RED (OpenJS Foundation): <https://nodered.org/>
- Broker público test.mosquitto.org: <https://test.mosquitto.org/>
- Broker alternativo HiveMQ: <https://www.hivemq.com/mqtt/public-mqtt-broker/>

12.4 ESP32 y hardware

- PlatformIO para ESP32: <https://docs.platformio.org/en/latest/boards/espressif32/esp32dev.html>
- Pinout ESP32 DevKit V1 (Espressif): <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/hw-reference/esp32/get-started-devkitc.html>
- Librería PubSubClient (MQTT para Arduino): <https://pubsubclient.knolleary.net/api>

- Librería Adafruit SSD1306 (pantalla OLED):
https://github.com/adafruit/Adafruit_SSD1306
- ArduinoJson v7 (parseo de JSON en ESP32):
<https://arduinojson.org/v7/tutorial/deserialization/>
- Blink Without Delay (patrón millis() de Arduino):
<https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay/>

12.5 MCP (Model Context Protocol)

- Especificación MCP (Anthropic): <https://modelcontextprotocol.io/>
- Paquete NuGet ModelContextProtocol:
<https://www.nuget.org/packages/ModelContextProtocol>

12.6 Accesibilidad y formato del documento

- British Dyslexia Association -- Guía de estilo:
<https://www.bdadyslexia.org.uk/advice/employers/creating-a-dyslexia-friendly-workplace/dyslexia-friendly-style-guide>
- WCAG 2.1 -- Pautas de accesibilidad web (W3C):
<https://www.w3.org/WAI/WCAG21/>
- WCAG 2.1 SC 1.4.8 -- Presentación visual:
<https://www.w3.org/WAI/WCAG21/Understanding/visual-presentation.html>
- WCAG 2.1 SC 1.4.12 -- Espaciado de texto:
<https://www.w3.org/WAI/WCAG21/Understanding/text-spacing.html>
- National Autistic Society -- Comunicación:
<https://www.autism.org.uk/advice-and-guidance/topics/communication>
- Atkinson Hyperlegible (Braille Institute):
<https://brailleinstitute.org/freefont>

- Rello, L. y Baeza-Yates, R. (2013). Good Fonts for Dyslexia. Proceedings of ASSETS '13:
<https://dl.acm.org/doi/10.1145/2513383.2513447>
- CAST -- Diseño Universal para el Aprendizaje (UDL):
<https://udlguidelines.cast.org/>